Smart Learning & AI Media OS — Step-by-Step Build Plan

Goal: a fully functional, usable application. No revenue targets, no business plan — pure execution: what to do, how to do it, where to do it, in order, for a 3-person team building simultaneously.

Total timeline: ~34 weeks (~8 months) to get all 24 modules live, working in parallel. Each phase below ends with a working, testable version of the product — you're never more than a few weeks from something you can actually click through.

Team Roles (fixed for the whole project)

Person	Codename	Owns
Person A	Backend/Infra Lead	Database, auth, API routes, DevOps, deployments, code-execution sandbox
Person B	Frontend/UX Lead	React app, video player, all UI screens, design system
Person C	AI/ML Lead	Every LLM-powered feature: summarizer, doubt bot, quiz gen, JAM evaluation, resume tailoring

Keep these lanes fixed for the whole build — switching lanes constantly is what causes 3-person teams to stall. Where a task needs two people (e.g., wiring an AI feature into the UI), that's called out explicitly below.

PHASE 0 — Environment & Project Setup (3–5 days, all 3 together)

Goal: everyone can run the project locally and push code by end of this phase.

Step 1: Accounts to create (do this first, together, one screen-share session)

1. **GitHub** — create an Organization (not personal repo), add all 3 as owners
2. **MongoDB Atlas** — free account → create a free M0 cluster → note the connection string
3. **Cloudinary** — free account → note Cloud Name, API Key, API Secret
4. **OpenAI** — API account → generate API key
5. **Anthropic Console** (console.anthropic.com) — API account → generate API key
6. **Render** or **Railway** — account for later deployment (not used yet, just create it now so billing/verification isn't a blocker later)
7. Pick one shared password manager (Bitwarden/1Password) to share these keys — never paste API keys in Slack/WhatsApp

Step 2: Local machine setup (each person, individually)

```bash
# Install Node.js LTS (v20+) and pnpm
node -v          # confirm v20+
npm install -g pnpm

# Install Docker Desktop (needed later for code-execution sandbox)
# Install VS Code or Antigravity IDE
# Install Claude Code CLI:
npm install -g @anthropic-ai/claude-code
```

Step 3: Repo scaffold (Person A does this, others clone after)

```bash
mkdir smart-learning-os && cd smart-learning-os
git init
pnpm init

# Monorepo structure
mkdir -p apps/web apps/api packages/shared-types packages/ui
```

Create `pnpm-workspace.yaml` at root:

```yaml
packages:
  - "apps/*"
  - "packages/*"
```

```bash
# Frontend scaffold
cd apps/web
pnpm create vite . --template react-ts
pnpm add tailwindcss @tailwindcss/vite zustand @tanstack/react-query axios reac

# Backend scaffold
cd ../../apps/api
pnpm init
pnpm add express mongoose dotenv cors helmet bcryptjs jsonwebtoken zod
pnpm add -D typescript ts-node-dev @types/express @types/node
npx tsc --init
```

Step 4: Environment variables

Create `apps/api/.env` (never commit this — add to `.gitignore` immediately):

```
MONGO_URI=<from Atlas>
JWT_SECRET=<random 32-char string>
JWT_REFRESH_SECRET=<another random string>
CLOUDINARY_CLOUD_NAME=<...>
CLOUDINARY_API_KEY=<...>
CLOUDINARY_API_SECRET=<...>
OPENAI_API_KEY=<...>
ANTHROPIC_API_KEY=<...>
PORT=5000
```

Step 5: CI setup (Person A)

Create `.github/workflows/ci.yml` — run lint + typecheck + build on every PR to `main`. Keep it simple at first: just `pnpm install && pnpm build` across both apps failing the check on error.

Step 6: Branching rules (agree as a team, write it in the README)

- `main` = always deployable
- Feature branches: `feat/<short-name>`, e.g. `feat/auth`, `feat/video-upload`
- PRs merge within 1–3 days max, reviewed by at least one other person
- No AI-generated PR merges without a human reading the diff first

End of Phase 0 checkpoint: all 3 people can run `pnpm dev` in both `apps/web` and `apps/api`, hit a `/health` endpoint from the frontend, and push a branch that passes CI.

PHASE 1 — Foundation: Auth + Dual Workspace + Data Model (Weeks 1–3)

Goal: a user can register, log in, land in their Private Workspace, and an admin can create an Org that others join.

Person A — Backend

Week 1: Core schemas. Create `apps/api/src/models/`:

javascript

```
// User.js
{
  name: String,
  email: { type: String, unique: true },
  passwordHash: String,
  role: { type: String, enum: ['student', 'instructor', 'org_admin'], default:
  currentOrgId: { type: ObjectId, ref: 'Organization', default: null },
  createdAt: Date
}

// Organization.js
{
  name: String,
  domain: String,           // for SSO/email-domain matching later
  members: [{ userId: ObjectId, role: String }],
  createdAt: Date
}

// PrivateWorkspace.js  — auto-created per user
{
  userId: { type: ObjectId, ref: 'User', unique: true },
  notes: [ObjectId],        // refs to Note docs, added later
  codeSandboxes: [ObjectId],
  createdAt: Date
}
```

Week 1–2: Auth routes (`/api/v1/auth`):

- `POST /register` → hash password (bcrypt), create User + auto-create PrivateWorkspace, return JWT
- `POST /login` → verify password, return access token (15 min) + refresh token (7 days, httpOnly cookie)
- `POST /refresh` → rotate access token
- `POST /logout`
- Middleware: `requireAuth`, `requireRole(['org_admin'])`

Week 2–3: Workspace routing

- `POST /api/v1/workspaces/switch` → sets `currentOrgId` on the user (or null for personal), returns which workspace context to load
- `POST /api/v1/orgs` (org_admin only) → create an org
- `POST /api/v1/orgs/:id/invite` → invite by email (simple: generate an invite link/token for MVP, real email sending can wait)

Person B — Frontend

Week 1: Design system. Set up Tailwind config with your color tokens/typography (this is the one place worth spending real design time — it's what makes the whole app feel "futuristic" instead of generic). Build reusable primitives first: `Button`, `Card`, `Modal`, `Sidebar`, `Avatar`.

Week 2: Auth screens — Login, Register, forgot-password (UI only, backend can stub it), protected route wrapper using the JWT in memory + refresh flow.

Week 3: Dual Workspace shell — a persistent top-level switcher (Personal ↔ Org) in the nav, and two distinct layout shells (Personal Workspace has a sidebar with Notes/Sandboxes/Resume/JAM History; Org Workspace has a sidebar with Courses/Cohort/Announcements). This shell is what every future feature gets mounted into, so get the layout right now.

Person C — AI/ML Lead

Phase 1 has no AI work yet. Use these 3 weeks to:

- Set up the **LLM Router abstraction** in `apps/api/src/services/ai/` — one function per capability (`summarize()`, `answerDoubt()`, `generateQuiz()`, `evaluateSpeech()`, `tailorResume()`), each internally choosing which provider/model to call. Build this now with stub implementations so Phase 2 just fills in the real logic.
- Prototype the transcription pipeline standalone: take one sample video, extract audio (ffmpeg), send to Whisper API, get back a timestamped transcript. Get this working as a script before it needs to be a service.

End of Phase 1 checkpoint: register → login → land in Personal Workspace → create an Org → switch into it → see an empty Org shell. Deploy this to Render/Railway now (even though it does almost nothing) — get the deployment pipeline working early, not at the end.

PHASE 2 — Core Learning Loop (Weeks 4–10)

Goal: upload a video → get AI notes → ask the AI questions about it → code alongside it → get quizzed → see it on a dashboard. This is the heart of the product.

Person A — Backend

Week 4–5: Video pipeline

- `POST /api/v1/videos/upload` → stream to Cloudinary, get back HLS manifest URL, save `Video` document (title, url, orgId/userId, duration, status: 'processing')
- Set up **BullMQ + Redis** (run Redis via Docker locally: `docker run -p 6379:6379 redis`) — on upload, enqueue a `processVideo` job
- Worker (can live in `apps/api/src/workers/`): pulls audio → sends to Whisper (Person C's script, now wired in) → saves transcript → triggers AI summarizer job → updates `Video.status = 'ready'`

Week 6–7: Coding Playground backend

- Set up **Judge0** via Docker Compose (`docker-compose.yml` with the official Judge0 image) — this gives you a REST API for sandboxed code execution, don't build your own sandbox
- `POST /api/v1/code/execute` → proxies to Judge0 with language + source + stdin, returns stdout/stderr/execution time
- Support JavaScript and Python first (2 languages only for MVP scope)

Week 8–10: Quiz + Dashboard backend

- `Quiz` schema: questions array (MCQ/fill-blank/code-challenge), linked to a videoId
- `POST /api/v1/videos/:id/quiz` → triggers Person C's quiz-gen AI function, saves result
- `Attempt` schema to store user quiz attempts + scores
- `GET /api/v1/analytics/me` → aggregate: videos watched, quiz scores, coding accuracy → basic dashboard data

Person B — Frontend

Week 4–5: Video player

- Integrate `hls.js` with a custom control bar (play/pause, speed 0.5x–2.5x, seek, chapter markers once available)
- Build the "processing" state UI (video uploaded but AI notes not ready yet — show a progress indicator, poll or use a websocket)

Week 6: Notes panel — renders the AI-generated markdown summary next to the video (split-pane layout)

Week 7–8: Coding Playground UI

- Monaco Editor mounted beside the video player, language selector, "Run" button hitting Person A's `/code/execute`, output console panel

Week 9: Doubt Assistant UI

- Chat panel (mounted in the same split-pane layout) — sends question + videoId to Person C's RAG endpoint, streams the response if possible (nicer UX than waiting for the full response)

Week 10: Quiz UI + Dashboard UI

- Quiz-taking flow (question → answer → next → results screen)
- Dashboard: simple charts (use `recharts`) for videos watched, quiz accuracy, coding streak

Person C — AI/ML Lead

Week 4–5: Summarizer — `summarize(transcript)` → prompts the LLM to output structured markdown (headings, key concepts, bullet takeaways) + a JSON list of chapter

timestamps. Test against 3–5 real lecture transcripts and tune the prompt until output is consistently well-structured, not just "usually good."

Week 6–7: RAG Doubt Assistant

- Chunk transcripts, embed with an embedding model (OpenAI `text-embedding-3-small` is a solid cheap default), store vectors in MongoDB Atlas Vector Search
- `answerDoubt(question, videoId)` → retrieve top-k relevant transcript chunks → pass as context to the LLM with the question → return grounded answer (and ideally the timestamp the answer came from, so the frontend can offer a "jump to this point" link)

Week 8–9: Quiz Generator — `generateQuiz(transcript)` → prompt the LLM for structured JSON output (MCQs with 4 options + correct answer + explanation, fill-in-blanks, and 1–2 code challenges). Force strict JSON output (use JSON mode / function-calling if your provider supports it) so the backend never has to parse loose text.

Week 10: Prompt QA pass — sit with Person B, run through the full loop end-to-end on 5 different real videos, fix whatever breaks (bad JSON, hallucinated timestamps, low-quality summaries).

End of Phase 2 checkpoint: a real user can upload a video, wait for processing, read AI notes, ask the AI a question about the content, write and run code in the playground, take an AI-generated quiz, and see it reflected on their dashboard. This is your first genuinely "wow, this works" milestone — deploy and use it yourselves for real content before moving on.

PHASE 3 — Practice & Career Modules (Weeks 11–18)

Goal: JAM Sessions, Resume Builder, Mock Interviews, Skill Gap Analyzer, Roadmap Engine, Portfolio Generator.

Person A — Backend

Week 11–12: JAM Session infra

- `JamSession` schema (topic, audio/video file ref, transcript, scores, feedback, userId, createdAt)
- `POST /api/v1/jam-sessions/start` → returns a random topic prompt
- `POST /api/v1/jam-sessions/evaluate` → accepts uploaded audio/video (store via Cloudinary), enqueue a job: transcribe → hand off to Person C's evaluation function → save results

Week 13–14: Resume Builder infra

- `Resume` schema (rawContent, targetCompany, targetRole, tailoredContent JSON, atsScore)
- `POST /api/v1/resumes/tailor` → accepts raw resume text + target company/role → calls Person C's tailoring function → returns structured tailored resume + ATS match %

Week 15–16: Mock Interview + Skill Gap

- `MockInterview` schema (questions asked, answers, per-question grading, overall score)
- `POST /api/v1/interviews/start` and `/api/v1/interviews/answer` (turn-based flow)
- `SkillGap` logic: compare a user's quiz/interview scores against a target role's expected skill list (seed a static skill-matrix JSON per role for MVP — don't over-engineer this with a live job-market scraper yet)

Week 17–18: Roadmap + Portfolio

- `POST /api/v1/roadmap/generate` → feeds skill-gap output into Person C's LLM call → returns a step-by-step ordered module list
- `Project` schema for the Portfolio Generator (suggested project title, description, tech stack, status: suggested/in-progress/done)

Person B — Frontend

Week 11–12: JAM Session UI — topic display, WebRTC/`MediaRecorder` browser API to record audio+video, upload flow, results card (fluency rate, filler-word count, feedback text) with playback of the recording, saved into Private Workspace history.

Week 13–14: Resume Builder UI — form for raw resume + company/role picker, side-by-side "before/after" view of tailored bullet points, ATS score meter, missing-keywords list.

Week 15–16: Mock Interview UI — chat-like turn-based interview flow, per-question feedback shown after each answer, final report screen.

Week 17–18: Roadmap + Portfolio UI — visual step-by-step roadmap (timeline/tree layout), project suggestion cards with a "start this" action that links back to relevant course videos.

Person C — AI/ML Lead

Week 11–12: JAM evaluation logic — from the transcript + audio duration, compute WPM, filler-word ratio (count "um/ah/like/you know" etc.), and prompt the LLM to grade grammar/technical accuracy against the topic, returning structured scores + written feedback.

Week 13–14: Resume tailoring logic — RAG or static company-profile data (build a small internal dataset of common company values/keywords per target company/role — don't try to live-scrape company sites for MVP) → prompt LLM to rewrite bullets into the Accomplished-[X]-as-measured-by-[Y]-by-doing-[Z] format, and compute a keyword-match percentage against the target role's expected keyword list.

Week 15–16: Mock interview logic — turn-based prompting: LLM asks a question relevant to the target role, grades the user's answer, decides the next question based on performance (adaptive difficulty is a nice touch but not required for v1 — a fixed question bank per role is a fine fallback).

Week 17–18: Roadmap generation logic — prompt the LLM with the user's skill-gap data + available course catalog to produce an ordered, personalized module sequence with reasoning for each step.

End of Phase 3 checkpoint: a user can record a JAM session and get real feedback, tailor a resume to a specific company, do a mock interview, and see a personalized roadmap. These are your most differentiated features — spend real testing time here, since low-quality AI feedback on career-critical content (resumes, interviews) will damage trust fast.

PHASE 4 — Collaboration & Knowledge Tools (Weeks 19–25)

Goal: Group Study Rooms, Mind Maps, Flashcards, Semantic Search, AI Animated Teaching Engine.

Person A — Backend

Week 19–21: Group Study Rooms

- This is the most infra-heavy remaining module. Use a managed WebRTC SFU — **LiveKit** (self-hostable or cloud) is the pragmatic choice over building your own with raw `mediasoup`.
- `StudyRoom` schema (participants, whiteboard state, createdAt)
- Socket.io for room presence, chat, and whiteboard event sync (cursor positions, shapes drawn) — LiveKit handles the actual audio/video/screen-share.

Week 22–23: Semantic Search backend

- Reuses the transcript vectors already built in Phase 2 — `POST /api/v1/search` embeds the user's natural-language query and does a vector similarity search across their accessible videos, returning ranked timestamp matches.

Week 24–25: Flashcards + supporting infra

- `Flashcard` schema with spaced-repetition metadata (interval, ease factor, next review date — implement the SM-2 algorithm, it's well documented and simple)
- `GET /api/v1/flashcards/due` → returns cards due for review today

Person B — Frontend

Week 19–21: Study Room UI — LiveKit's React SDK handles most of the video-grid UI; you build the shared whiteboard canvas (use `Konva.js` or `Fabric.js` for canvas drawing) and the room chat panel.

Week 22: Semantic Search UI — a search bar that returns a list of timestamped results across the user's videos, click-through jumps straight to that point in the player.

Week 23–24: Mind Map UI — render the LLM-generated node tree (Person C builds this below) using a library like `react-flow` — interactive, draggable nodes.

Week 25: Flashcards UI — flip-card review interface, "due today" queue, simple rating buttons (Again/Hard/Good/Easy) feeding back into the SM-2 scheduler.

Person C — AI/ML Lead

Week 19–22: Mind Map generation — prompt the LLM to output a structured JSON node tree (concept → sub-concepts → relationships) from a transcript; this is a pure prompt-engineering task, no new infra needed.

Week 23–24: Flashcard generation — prompt the LLM to produce question/answer pairs from transcript content, tuned for spaced-repetition style (atomic, single-concept cards, not paragraph answers).

Week 25: AI Animated Teaching Engine (start) — this is the most experimental module. Approach: prompt the LLM to output a *declarative animation spec* (e.g., a sequence of steps describing what to draw/highlight/move on a canvas — not raw animation code), then Person B builds a renderer that interprets that spec with `Konva.js`/`Framer Motion`. Scope this down for v1 to one use case (e.g., visualizing a sorting algorithm or a code execution trace) rather than trying to generalize to "any concept" immediately.

End of Phase 4 checkpoint: users can study together in real-time rooms, search across all their video content semantically, review AI-generated flashcards, and see at least one working animated concept visualization.

PHASE 5 — Org Features, Analytics, Notifications, Certificates, Offline (Weeks 26–30)

Goal: everything that makes this usable by a real cohort/org, not just an individual.

Person A — Backend

- **Week 26–27:** Org Cohort Analytics — aggregate queries across all members of an org (avg quiz scores, video completion rates, skill-gap distribution) → `GET /api/v1/orgs/:id/analytics`
- **Week 28:** Certificate generation — on course completion, generate a signed PDF (use a library like `pdf-lib`) with a verifiable QR code linking to a public verification endpoint (`GET /api/v1/certificates/verify/:id`)
- **Week 29:** Notification Engine — start simple: in-app notification feed (`Notification` schema + `GET /api/v1/notifications`) triggered by events (quiz due, JAM streak reminder, deadline approaching). Email notifications (via Resend/SendGrid) can be layered on afterward.
- **Week 30:** Offline Sync — service worker + IndexedDB (via a library like `Dexie.js`) caching notes, transcripts, flashcards, and quiz data for offline access. Explicitly do NOT cache video files locally (storage + licensing complexity) — only text-based content.

Person B — Frontend

- **Week 26–27:** Org Admin dashboard — cohort-level charts, member list, role management UI
- **Week 28:** Certificate UI — a shareable certificate page + download button

- **Week 29:** Notification bell/feed component in the main nav
- **Week 30:** Offline mode indicator + service worker registration, "available offline" badges on content

Person C — AI/ML Lead

- **Week 26–28:** Org-level Skill Gap Analysis — extend the individual skill-gap logic to aggregate across a cohort and highlight common weak areas (useful output for instructors)
- **Week 29–30:** Notification content generation — personalized nudge copy (e.g., "You're 80% through Data Structures, one more quiz to finish") generated by a lightweight LLM prompt rather than static templates, plus general QA pass across every AI feature built so far — re-test with fresh sample data across all modules.

End of Phase 5 checkpoint: the full 24-module product is functionally complete.

PHASE 6 — Testing, Hardening, Deployment (Weeks 31–34)

Goal: ship it somewhere real people can use it.

Week 31 — Security pass (Person A leads, everyone reviews)

- Audit the code-execution sandbox specifically: confirm no network access from executed code, resource/time limits enforced, no filesystem escape
- Rate limiting on all AI endpoints (LLM calls cost money and can be abused)
- Input validation (`zod`) on every route, not just the happy path
- Rotate all API keys that were shared during setup, move to a proper secrets manager (Render/Railway env vars, not `.env` files, in production)

Week 32 — Load & QA pass (all 3)

- Manually walk every one of the 24 modules end-to-end, write down every bug
- Basic load test on the video streaming + code execution endpoints (these are your two most resource-heavy paths)
- Fix P0/P1 bugs; log P2/P3 for post-launch

Week 33 — Deployment (Person A leads)

1. **Backend:** deploy `apps/api` to Render/Railway (or migrate to AWS ECS/Fargate if you're already outgrowing PaaS) — set all production env vars, enable auto-deploy from `main`
2. **Frontend:** deploy `apps/web` to Vercel or Netlify (fastest path for a Vite/React app)
3. **Database:** upgrade MongoDB Atlas from free M0 to a paid tier with backups enabled
4. **Redis:** managed Redis (Upstash is a simple serverless option) instead of self-hosted
5. **Judge0/code execution:** deploy on a small dedicated VM or container service — this needs isolation from your main API infra
6. **LiveKit:** use LiveKit Cloud rather than self-hosting the SFU initially — self-hosting WebRTC infra is a real ops burden you don't need yet

7. **Domain + SSL:** point your domain at the frontend deploy, backend gets its own subdomain (`api.yourapp.com`), SSL is automatic on Vercel/Render
8. Set up **Sentry** for error tracking and **UptimeRobot/Better Uptime** for uptime monitoring in production — do this before, not after, real users show up

Week 34 — Soft launch

- Onboard a small real group (classmates, a bootcamp cohort, your own friends) as first real users
- Watch Sentry + analytics closely for the first week, fix what breaks

---

Reference: Full Tech Stack

Layer	Choice
Frontend	React + Vite, TailwindCSS, Zustand, TanStack Query, react-flow, Konva.js
Video	Cloudinary (HLS) + hls.js
Code Editor/Exec	Monaco Editor + Judge0 (Dockerized)
Backend	Node.js + Express + TypeScript
Database	MongoDB Atlas (+ Vector Search)
Cache/Queue	Redis + BullMQ (Upstash in production)
Real-time	Socket.io + LiveKit (WebRTC SFU)
Transcription	Whisper API
LLM	Claude API + OpenAI API behind an internal router
Auth	JWT + refresh tokens, OAuth2 for institutional SSO
Hosting	Render/Railway (API) + Vercel (frontend) → AWS if you outgrow it
Monitoring	Sentry + UptimeRobot

Reference: AI Tool Split (Antigravity / Claude Code / ChatGPT / Gemini)

- **Antigravity:** scaffolding new modules, parallel feature builds (its Manager view maps onto your 3-lane team split), browser-verified UI work
- **Claude Code:** multi-file, careful work — auth/RBAC, the LLM router, the code-execution sandbox, and any refactor that must stay consistent across many files
- **ChatGPT/Gemini:** fast single-file debugging, prompt brainstorming for the AI features, quick second opinions — not large multi-file changes
- Every AI-generated PR still gets a human read before merge, every phase, no exceptions